（五）实战项目：企业合同智能审查 Hierarchical 多 Agent 系统

项目背景

在企业日常经营过程中，法务、商务、采购、人力资源等部门经常需要处理大量合同，例如**房屋租赁合同、劳务合同、设备租赁合同、采购合同、服务合同**等。传统合同审查主要依赖人工完成，通常存在以下问题：

- **审查耗时较长**：合同篇幅较长，条款细节多，人工逐条审查效率较低；
- **风险容易遗漏**：付款、违约、解除、争议解决等关键条款容易因人工疲劳或经验差异被忽略；
- **审查标准不统一**：不同法务人员对风险等级、修改建议的判断可能不一致；
- **报告输出不规范**：人工审查结果常以批注或文字说明呈现，缺少统一结构，不便于业务部门理解和执行；
- **修改建议不够落地**：很多审查意见只指出问题，但没有直接给出可替换的修改后条款。

本项目基于 **CrewAI Hierarchical 多 Agent 架构**，构建一个面向企业场景的**合同智能审查助手。**系统通过一个专门的 **Manager Agent** 作为项目经理，统筹调度多个专业 Agent，完成从合同读取、文本清洗、风险识别、条款修改、终审复核到报告保存的完整流程。

与普通 sequential 顺序执行项目不同，本项目重点体现 **Hierarchical Process** 的多 Agent 协作能力：由 Manager Agent 作为中心调度者，负责理解整体任务目标、协调不同专家 Agent 执行子任务，并确保最终交付结果满足企业级合同审查要求。

项目核心价值：

- 使用 **Hierarchical 多 Agent 工作流**模拟真实企业法务团队协作流程；
- 通过 **Manager Agent** 统一调度合同读取、风险审核、条款修改和终审交付任务；
- 支持 **PDF、TXT、Markdown** 多格式合同文件读取；
- 自动完成合同文本清洗，尽量保留条款编号、标题和段落结构；
- 自动识别合同中的法律风险、商业风险、合规风险和履约风险；
- 按照高 / 中 / 低风险等级输出结构化风险点清单；
- 针对风险条款生成具体、可执行的修改建议；
- 重点生成**修改前后的条文对比表**，提升审查结果的可落地性；
- 自动生成 Markdown 格式合同审查报告；
- 自动保存最终报告和修改后条文文件，形成完整项目交付物。

1.项目讲解

1.1 项目团队

本项目采用 **1 个 Manager Agent + 4 个专业执行 Agent** 的 Hierarchical 协作模式，共包含 5 个 Agent。

Agent 团队组成

Agent 名称	中文角色	核心职责	是否允许任务委派
Manager Agent	企业合同智能审查项目经理	统筹任务、分配工作、协调流程、控制交付质量	是
Cleaner Agent	合同读取与文本清洗专家	读取合同文件，清洗合同文本，保留条款结构	否
Reviewer Agent	合同法律与商业风险审核专家	识别合同风险，输出风险等级和风险说明	否
Modifier Agent	合同条款修改与优化专家	针对风险点生成修改建议和前后条文对比	否
Validator Agent	合同审查终审与报告交付专家	复核整体结果，生成并保存最终审查报告	否

该团队设计模拟了真实企业合同审查中的岗位分工：

```
项目经理 / 法务负责人
        ↓
文档处理专员 → 合同审核律师 → 合同修改律师 → 法务总监终审
```

其中，Manager Agent 不直接承担具体合同条款审查工作，而是负责**任务调度与流程管理**，体现 Hierarchical 多 Agent 系统的核心特征。

1.2 团队分工

1.2.1 企业合同智能审查项目经理（Manager Agent）

Manager Agent 是本项目的核心调度角色，也是区别于 sequential 项目的关键设计。

（1）主要职责

Manager Agent 负责统筹整个合同审查流程，包括：

- 理解用户输入的合同审查需求；
- 接收用户传入的合同文件路径；
- 根据任务目标协调不同专业 Agent；
- 将合同读取与清洗任务分配给 Cleaner Agent；
- 将风险识别任务分配给 Reviewer Agent；
- 将条款修改任务分配给 Modifier Agent；
- 将终审复核和报告保存任务分配给 Validator Agent；
- 检查各阶段输出是否完整；
- 确保最终报告包含所有指定模块；

- 保障最终结果符合企业合同审查交付标准。

(2) Manager Agent 的技术设置

在 CrewAI 中，本项目使用 `Process.hierarchical` 启用层级式多 Agent 协作，并通过 `manager_agent` 参数指定项目经理 Agent。

Manager Agent 设置为：

```
manager_agent = Agent(
    role="企业合同智能审查项目经理",
    goal=(
        "作为 hierarchical 多 Agent 系统的总协调者，负责理解用户输入的合同审查需求，"
        "合理分配文档读取清洗、风险识别、条款修改、终审报告生成等任务，"
        "监控各 Agent 的执行质量，确保最终交付结构化、专业化、可落地的合同审查报告。"
    ),
    backstory=(
        "你是一位拥有多年企业法务项目管理经验的合同审查项目经理，"
        "熟悉企业合同审查的完整业务流程，包括合同文本解析、关键条款提取、法律风险识别、"
        "商业风险判断、修改建议制定、终审复核和审查报告交付。"
    ),
    verbose=True,
    allow_delegation=True
)
```

其中最关键的是：

```
allow_delegation=True
```

这表示 Manager Agent 具备任务委派能力，可以在 Hierarchical 流程中协调其他 Agent 完成任务。

(3) Manager Agent 不放入普通执行 Agent 列表

为了避免 Manager 既作为管理者又作为普通执行者造成职责混乱，本项目在 Crew 配置中采用如下方式：

```
crew = Crew(
    agents=[
        cleaner_agent,
        reviewer_agent,
        modifier_agent,
        validator_agent
    ],
    tasks=[
        task1_clean,
        task2_review,
        task3_modify,
        task4_validate
    ],
    process=Process.hierarchical,
    manager_agent=manager_agent,
    verbose=True
)
```

这里需要注意：

- `manager_agent=manager_agent` 单独指定 Manager；
- `agents=[...]` 中只放入具体执行任务的专业 Agent；
- Manager Agent 不再重复放入普通 agents 列表；
- 这样可以让 Manager 专注于调度，而不是参与具体执行。

这种设计更符合 Hierarchical 架构，也更接近真实企业项目管理流程。

1.2.2 合同读取与文本清洗专家（Cleaner Agent）

Cleaner Agent 负责合同审查流程的第一步：**读取合同文件并清洗文本。**

（1）主要职责

Cleaner Agent 需要完成：

- 根据用户指定路径读取合同文件；
- 支持 PDF、TXT、Markdown 三类文件格式；
- 对 PDF 文件使用 `PyMuPDFLoader` 提取文本；
- 对 TXT 和 Markdown 文件使用本地文件读取；
- 兼容中文文本常见编码问题，例如 UTF-8 和 GBK；
- 清除乱码、零宽字符、特殊符号、页眉页脚；
- 保留合同标题、条款编号和自然段落；
- 输出结构清晰、适合后续审查的合同全文。

（2）对应工具

Cleaner Agent 绑定了两个工具：

```
tools=[load_contract_file, clean_contract_text]
```

其中：

- `load_contract_file` 负责读取合同文件；
- `clean_contract_text` 负责合同文本清洗。

（3）文件读取工具能力

本项目中的文件读取工具支持根据文件后缀自动选择读取方式：

```
ext = os.path.splitext(file_path)[1].lower()
```

支持格式包括：

文件格式	处理方式
PDF	使用 `PyMuPDFLoader` 提取文本

文件格式	处理方式
TXT	使用本地文件读取
Markdown	使用本地文件读取

例如：

```
if ext == ".pdf":
    from langchain_community.document_loaders import PyMuPDFLoader
    loader = PyMuPDFLoader(file_path)
    docs = loader.load()
    text = "\n".join([doc.page_content for doc in docs])
```

对于 TXT 和 Markdown 文件，则使用：

```
with open(file_path, "r", encoding="utf-8") as f:
    return f.read().strip()
```

如果 UTF-8 读取失败，会自动尝试 GBK 编码：

```
except UnicodeDecodeError:
    with open(file_path, "r", encoding="gbk") as f:
        return f.read().strip()
```

这对于中文合同文件非常实用。

(4) 文本清洗策略

合同文本不同于普通文章，不能简单把所有空白压缩成一行。因此本项目的清洗策略强调：

- 保留换行；
- 保留条款层级；
- 保留段落结构；
- 只清除噪声内容。

主要清洗内容包括：

- 零宽字符；
- BOM 字符；
- 特殊乱码符号；
- 页眉页脚；
- 多余空格；
- 过长分隔线；
- 连续空行。

清洗后的合同文本会作为后续 Reviewer Agent 的输入基础。

1.2.3 合同法律与商业风险审核专家（Reviewer Agent）

Reviewer Agent 是合同审查流程中的核心业务 Agent，负责从清洗后的合同文本中识别风险。

（1）主要职责

Reviewer Agent 需要从多个维度进行合同审查，包括：

- 法律风险；
- 商业风险；
- 合规风险；
- 履约风险；
- 条款缺失风险；
- 表述不明确风险。

（2）审查维度

本项目要求 Reviewer Agent 至少从以下维度进行审查：

1. 合同主体是否明确；
2. 签约主体是否具备履约能力；
3. 合同标的、服务内容、租赁物或工作内容是否具体；
4. 合同金额、付款方式、付款节点、发票、押金或保证金是否明确；
5. 履行期限、交付时间、服务周期、租赁期限是否清晰；
6. 验收标准、交付标准、质量标准是否明确；
7. 双方权利义务是否平衡；
8. 违约责任是否具体、可执行；
9. 合同解除、终止、提前退租或提前终止机制是否公平；
10. 保密、知识产权、数据安全、资料返还等条款是否完整；
11. 不可抗力、通知送达、合同变更等一般条款是否完备；
12. 争议解决方式、管辖法院或仲裁机构是否明确；
13. 是否存在明显不公平或表述模糊的条款。

（3）风险等级设计

Reviewer Agent 需要按照高 / 中 / 低三个等级进行风险分类。

风险等级	定义	示例
高风险	可能导致重大经济损失、核心权利无法保障或严重不利于企业	无违约责任、付款条件严重不清、争议解决缺失
中风险	可能导致履约不确定、争议成本增加，但可通过修改控制	验收标准不清、解除条件不完整
低风险	主要是格式、表述或一般信息不完整问题	条款表述不严谨、通知方式不够明确

（4）输出要求

Reviewer Agent 必须输出结构化风险点清单，推荐格式如下：

```
| 风险编号 | 风险等级 | 条款位置 | 原条款摘录 | 风险类型 | 风险描述 | 可能后果 | 初步修改方向 |
|---|---|---|---|---|---|---|---|
| R-001 | 高 | 第三条 付款方式 | …… | 商业风险 | …… | …… | …… |
```

这使得风险识别结果可以直接传递给 Modifier Agent 进行后续修改。

1.2.4 合同条款修改与优化专家（Modifier Agent）

Modifier Agent 负责将 Reviewer Agent 识别出的风险点转化为具体修改方案。

（1）主要职责

Modifier Agent 需要完成：

- 阅读风险点清单；
- 针对每个风险编号生成修改建议；
- 对高风险和中风险条款给出完整替代文本；
- 对低风险条款给出简洁优化建议；
- 生成修改前后条文对比表；
- 对合同缺失条款提供建议新增条款。

（2）修改原则

Modifier Agent 的修改原则包括：

- 修改建议必须具体，不能只写“建议完善”；
- 修改后条款应使用正式合同语言；
- 修改后条款应可直接复制进合同；
- 修改理由要与风险点一一对应；
- 对付款、违约、解除、验收、争议解决等核心条款要提高可执行性；
- 修改建议要兼顾企业利益保护与商业合理性。

（3）输出结构

Modifier Agent 输出两个核心部分。

第一部分是修改建议列表：

```
| 建议编号 | 对应风险 | 修改方向 | 具体建议 | 修改理由 |
|---|---|---|---|---|
| M-001 | R-001 | 明确付款节点 | 建议补充付款时间、付款条件和逾期责任 | 降低回款争议 |
```

第二部分是修改前后的条文对比：

```
| 风险编号 | 条款位置 | 原条款 | 修改后条款 | 修改理由 |
|---|---|---|---|---|
| R-001 | 第三条 付款方式 | …… | …… | …… |
```

其中，**修改前后的条文对比**是本项目的重点输出，也是最能体现项目实用价值的部分。

1.2.5 合同审查终审与报告交付专家（Validator Agent）

Validator Agent 是整个系统的最终质量负责人，负责将前面多个 Agent 的输出整合为正式合同审查报告。

（1）主要职责

Validator Agent 需要完成：

- 复核 Cleaner Agent 的合同文本是否完整；
- 复核 Reviewer Agent 的风险点是否清晰；
- 复核 Modifier Agent 的修改建议是否可执行；
- 检查高风险和中风险是否都有对应修改方案；
- 检查修改前后条文是否一一对应；
- 生成完整 Markdown 合同审查报告；
- 单独整理修改后条文；
- 调用保存工具保存最终文件。

（2）绑定工具

Validator Agent 绑定了两个保存工具：

```
tools=[save_final_report, save_modified_clauses]
```

分别用于：

工具名称	作用	默认保存文件
`save_final_report`	保存最终合同审查报告	`outputs/contract_review_report.md`
`save_modified_clauses`	保存修改后的合同条文	`outputs/modified_clauses.md`

终审标准

Validator Agent 的终审标准包括：

1. 报告是否包含合同基本信息；
2. 是否提取关键条款；
3. 风险点是否包含风险等级、条款位置和风险描述；
4. 高风险和中风险是否都有修改建议；
5. 修改建议是否具体、可执行；

6. 修改前后条文是否一一对应；
7. 总体审查意见是否合理；
8. 是否包含 AI 辅助审查声明；
9. 是否成功保存报告和修改后条文文件。

1.3 项目技术架构

本项目采用 **CrewAI + Tools + Hierarchical Process** 的技术架构。

1.3.1 整体流程

```
用户输入合同文件路径
        ↓
Manager Agent 接收任务并统筹调度
        ↓
Cleaner Agent 读取并清洗合同文本
        ↓
Reviewer Agent 识别法律 / 商业 / 合规风险
        ↓
Modifier Agent 生成修改建议和前后条文对比
        ↓
Validator Agent 终审复核并生成报告
        ↓
保存 contract_review_report.md 和 modified_clauses.md
```

1.3.2 Hierarchical 架构关系

```
Manager Agent
|
├── Cleaner Agent
|   ├── load_contract_file
|   └── clean_contract_text
|
├── Reviewer Agent
|   └── 风险识别与风险评级
|
├── Modifier Agent
|   └── 修改建议与条文改写
|
└── Validator Agent
    ├── save_final_report
    └── save_modified_clauses
```

1.4 核心技术点解析

1.4.1 使用 `Process.hierarchical` 构建层级式多 Agent 工作流

本项目不是简单的 sequential 顺序执行，而是使用：

```
process=Process.hierarchical
```

Hierarchical 模式的特点是：

- 有一个 Manager Agent 负责全局调度；
- 其他 Agent 作为专业执行者；
- Manager 根据任务目标协调任务执行；
- 更接近真实企业中的“项目经理 + 专家团队”工作模式。

对应代码：

```
crew = Crew(
    agents=[
        cleaner_agent,
        reviewer_agent,
        modifier_agent,
        validator_agent
    ],
    tasks=[
        task1_clean,
        task2_review,
        task3_modify,
        task4_validate
    ],
    process=Process.hierarchical,
    manager_agent=manager_agent,
    verbose=True
)
```

这个配置体现了项目的 Multi-Agent 层级调度能力。

1.4.2 Manager Agent 专门负责任务分配

Manager Agent 的关键配置是：

```
allow_delegation=True
```

这表示 Manager 可以将任务分配给其他 Agent。

而其他 Agent 设置为：

```
allow_delegation=False
```

表示它们只负责自己的专业任务，不再继续向下委派。

这种设置让系统形成明确的层级关系：

```
Manager：负责分配任务
专业 Agent：负责执行任务
```

这也是本项目区别于普通多 Agent 串行流程的重要技术点。

1.4.3 Manager Agent 单独配置，不混入普通 Agent 列表

为了避免调度混乱，本项目采用：

```
manager_agent=manager_agent
```

单独配置项目经理 Agent。

普通执行 Agent 列表中只包含：

```
agents=[
    cleaner_agent,
    reviewer_agent,
    modifier_agent,
    validator_agent
]
```

不把 Manager Agent 放入普通 agents 列表。

这种设计可以避免：

- Manager 被当成普通执行 Agent；
- Manager 自己处理合同审查任务；
- Manager 与执行 Agent 职责重叠；
- 层级调度逻辑混乱。

1.4.4 使用 Tool 扩展 Agent 的实际操作能力

本项目中 Agent 不只是生成文本，还可以调用工具完成实际文件处理。

（1）工具一：合同文件读取工具

```
@tool
def load_contract_file(file_path: str) -> str:
    ...
```

功能包括：

- 检查文件是否存在；

- 判断文件格式；
- 读取 PDF；
- 读取 TXT；
- 读取 Markdown；
- 返回合同原始文本。

(2) 工具二：合同文本清洗工具

```
@tool
def clean_contract_text(raw_text: str) -> str:
    ...
```

功能包括：

- 去除零宽字符；
- 去除特殊符号；
- 去除页眉页脚；
- 清理多余空格；
- 保留段落结构；
- 保留合同条款编号。

(3) 工具三：报告保存工具

```
@tool
def save_final_report(report_content: str, filename: str = "contract_review_report.md")
-> str:
    ...
```

功能是将最终审查报告保存到：

```
outputs/contract_review_report.md
```

(4) 工具四：修改后条文保存工具

```
@tool
def save_modified_clauses(modified_clauses: str, filename: str = "modified_clauses.md")
-> str:
    ...
```

功能是将修改后的条文保存到：

```
outputs/modified_clauses.md
```

这些工具让 Agent 具备了真实项目中的文件读取、清洗和保存能力。

1.4.5 使用 Context 实现 Agent 之间的结果传递

本项目中，后续任务依赖前序任务的输出，因此使用 `context` 进行结果传递。

例如，风险审核任务依赖文本清洗结果：

```
task2_review = Task(
    ...,
    context=[task1_clean]
)
```

条款修改任务依赖风险审核结果：

```
task3_modify = Task(
    ...,
    context=[task2_review]
)
```

终审任务依赖前面三个阶段结果：

```
task4_validate = Task(
    ...,
    context=[task1_clean, task2_review, task3_modify]
)
```

这样形成了清晰的信息流：

```
清洗后的合同文本
        ↓
风险点清单
        ↓
修改建议和条文对比
        ↓
最终审查报告
```

Context 是多 Agent 协同中非常重要的机制，它解决了不同 Agent 之间“如何交接工作结果”的问题。

1.5 输入与输出要求

1.5.1 输入

本项目输入为一份合同文件，支持以下格式：

- PDF；
- TXT；
- Markdown。

文件路径由用户指定，例如：

```
contract_path = "contract_sample.pdf"
```

也可以指定为：

```
contract_path = "contracts/房屋租赁合同.txt"
contract_path = "contracts/劳务合同.md"
contract_path = "contracts/设备租赁合同.pdf"
```

启动时通过：

```
result = crew.kickoff(inputs={"file_path": contract_path})
```

将文件路径传递给任务。

在任务描述中使用：

```
用户提供了一份企业合同文件，文件路径为：{file_path}
```

确保 Agent 能够获取用户输入的文件路径。

1.5.2 输出

系统最终输出一份结构化 Markdown 合同审查报告。

报告必须包含：

1. 合同基本信息；
2. 关键条款提取表；
3. 风险点清单；
4. 修改建议列表；
5. 修改前后的条文对比；
6. 总体审查意见；
7. 修改后条文汇总；
8. AI 辅助审查声明。

1.5.3 保存要求

最终系统需要自动生成两个文件：

```
outputs/contract_review_report.md
outputs/modified_clauses.md
```

其中：

- `contract_review_report.md` 保存完整审查报告；

- `modified_clauses.md` 保存所有修改后条文或建议新增条款。

1.6 最终报告结构

最终报告采用统一 Markdown 模板，方便企业法务和商务部门阅读。

```
# 企业合同智能审查报告

## 一、合同基本信息
- 合同名称：
- 合同类型：
- 合同主体：
- 审查日期：
- 审查方式：CrewAI Hierarchical 多 Agent 智能审查

## 二、关键条款提取表
| 条款类别 | 条款位置 | 主要内容 | 审查关注点 |
|---|---|---|---|

## 三、风险点清单
| 风险编号 | 风险等级 | 条款位置 | 风险类型 | 风险描述 | 可能后果 |
|---|---|---|---|---|---|

## 四、修改建议列表
| 建议编号 | 对应风险 | 修改方向 | 具体建议 | 修改理由 |
|---|---|---|---|---|

## 五、修改前后的条文对比
| 风险编号 | 条款位置 | 原条款 | 修改后条款 | 修改理由 |
|---|---|---|---|---|

## 六、总体审查意见
### 1. 总体风险等级
### 2. 主要问题概述
### 3. 签署前建议

## 七、修改后条文汇总

## 八、AI 辅助审查声明
本报告由 AI 辅助生成，仅供企业内部初步审查参考，不构成正式法律意见。正式签署前，建议由专业律师或企业法务人员进行最终确认。
```

其中，**第五部分“修改前后的条文对比”**是本项目最重要的展示内容。

1.7 项目执行流程

1.7.1 第一步：用户指定合同路径

用户在主程序中指定合同文件：

```
contract_path = "contract_sample.pdf"
```

然后通过 `kickoff` 启动 Crew：

```
result = crew.kickoff(inputs={"file_path": contract_path})
```

1.7.2 第二步：Manager Agent 接收整体任务

Manager Agent 接收合同审查目标后，负责统筹任务执行。

它需要理解：

- 当前要审查哪份合同；
- 需要哪些专业 Agent 参与；
- 每个阶段的输出要求；
- 最终报告需要满足哪些结构要求。

1.7.3 第三步：Cleaner Agent 读取并清洗合同

Cleaner Agent 调用：

```
load_contract_file
```

读取合同内容，然后调用：

```
clean_contract_text
```

清洗合同文本。

输出内容包括：

- 文件读取状态；
- 清洗说明；
- 清洗后的合同正文。

1.7.4 第四步：Reviewer Agent 识别合同风险

Reviewer Agent 基于清洗后的合同文本进行风险审查，输出：

- 风险编号；
- 风险等级；
- 条款位置；
- 原条款摘录；
- 风险类型；
- 风险描述；
- 可能后果；
- 初步修改方向。

1.7.5 第五步：Modifier Agent 生成修改建议

Modifier Agent 基于风险清单输出：

- 修改建议列表；
- 修改前后的条文对比表；
- 建议新增条款；
- 修改理由。

1.7.6 第六步：Validator Agent 终审并保存结果

Validator Agent 综合前面三个阶段结果，生成完整合同审查报告，并调用工具保存：

```
save_final_report
save_modified_clauses
```

最终生成：

```
outputs/contract_review_report.md
outputs/modified_clauses.md
```

2.具体实现

```python
from crewai import Agent, Task, Crew, Process
from crewai.tools import tool
import re
import os


# ==================== 1. 定义工具 ====================
```

```python

@tool
def load_contract_file(file_path: str) -> str:
    """
    读取合同文件内容，支持 PDF、TXT、Markdown 格式。
    参数：
        file_path: 合同文件路径，例如 contract_sample.pdf / contracts/劳务合同.md
    返回：
        提取后的合同原始文本
    """
    try:
        if not os.path.exists(file_path):
            return f"文件读取失败：文件不存在：{file_path}"

        ext = os.path.splitext(file_path)[1].lower()

        if ext == ".pdf":
            from langchain_community.document_loaders import PyMuPDFLoader
            loader = PyMuPDFLoader(file_path)
            docs = loader.load()
            text = "\n".join([doc.page_content for doc in docs])
            return text.strip()

        elif ext in [".txt", ".md"]:
            try:
                with open(file_path, "r", encoding="utf-8") as f:
                    return f.read().strip()
            except UnicodeDecodeError:
                with open(file_path, "r", encoding="gbk") as f:
                    return f.read().strip()

        else:
            return f"文件读取失败：暂不支持该文件格式 {ext}，请使用 PDF、TXT 或 MD 文件。"

    except Exception as e:
        return f"文件读取失败: {str(e)}"


@tool
def clean_contract_text(raw_text: str) -> str:
    """
    对合同文本进行专业清洗。
    目标：
    1. 去除乱码、页眉页脚、特殊符号；
    2. 保留合同标题、条款编号、自然段落；
    3. 避免将整份合同压缩成一行。
    """
    try:
        if not raw_text or raw_text.strip() == "":
            return "文本清洗失败：输入文本为空。"

        text = raw_text

```

```python
        # 1. 去除零宽字符、BOM 等不可见字符
        text = re.sub(r'[\u200b\u200c\u200d\ufeff]', '', text)

        # 2. 去除常见特殊符号和乱码符号
        text = re.sub(r'[■◆□●★※]', '', text)

        # 3. 去除常见页眉页脚
        text = re.sub(r'第\s*\d+\s*页\s*/?\s*共\s*\d+\s*页', '', text)
        text = re.sub(r'第\s*\d+\s*页', '', text)
        text = re.sub(r'共\s*\d+\s*页', '', text)

        # 4. 去除常见页眉字段，但尽量不误删正文
        text = re.sub(r'^\s*合同编号[:： ].*$', '', text, flags=re.MULTILINE)
        text = re.sub(r'^\s*签署日期[:： ].*$', '', text, flags=re.MULTILINE)

        # 5. 按行处理，保留条款结构
        lines = text.splitlines()
        cleaned_lines = []

        for line in lines:
            line = line.strip()

            # 删除过长分隔线
            line = re.sub(r'[-_=]{5,}', '', line)

            # 标点前多余空格
            line = re.sub(r'\s+([，。；：？！、])', r'\1', line)

            # 多个连续空格压缩成一个，但不影响换行
            line = re.sub(r'[ \t]{2,}', ' ', line)

            cleaned_lines.append(line)

        # 6. 连续空行最多保留一个
        result_lines = []
        blank_count = 0

        for line in cleaned_lines:
            if line == "":
                blank_count += 1
                if blank_count <= 1:
                    result_lines.append(line)
            else:
                blank_count = 0
                result_lines.append(line)

        text = "\n".join(result_lines)

        return text.strip()

    except Exception as e:
        return f"文本清洗失败: {str(e)}"

```

```python

@tool
def save_final_report(report_content: str, filename: str =
"contract_review_report.md") -> str:
    """
    保存最终审查报告为 Markdown 文件。
    默认保存路径：outputs/contract_review_report.md
    """
    try:
        output_dir = "outputs"
        os.makedirs(output_dir, exist_ok=True)

        file_path = os.path.join(output_dir, filename)

        with open(file_path, "w", encoding="utf-8") as f:
            f.write(report_content)

        return f"审查报告已保存至：{file_path}"

    except Exception as e:
        return f"保存失败: {str(e)}"


@tool
def save_modified_clauses(modified_clauses: str, filename: str =
"modified_clauses.md") -> str:
    """
    保存修改后的合同条文。
    默认保存路径：outputs/modified_clauses.md
    """
    try:
        output_dir = "outputs"
        os.makedirs(output_dir, exist_ok=True)

        file_path = os.path.join(output_dir, filename)

        with open(file_path, "w", encoding="utf-8") as f:
            f.write(modified_clauses)

        return f"修改后的条文已保存至：{file_path}"

    except Exception as e:
        return f"保存失败: {str(e)}"


# ==================== 2. 定义五个 Agent ====================

manager_agent = Agent(
    role="企业合同智能审查项目经理",
    goal=(
        "作为 hierarchical 多 Agent 系统的总协调者，负责理解用户输入的合同审查需求，"
        "合理分配文档读取清洗、风险识别、条款修改、终审报告生成等任务，"
        "监控各 Agent 的执行质量，确保最终交付结构化、专业化、可落地的合同审查报告。"
```

```
    ),
    backstory=(
        "你是一位拥有多年企业法务项目管理经验的合同审查项目经理，"
        "熟悉企业合同审查的完整业务流程，包括合同文本解析、关键条款提取、法律风险识别、"
        "商业风险判断、修改建议制定、终审复核和审查报告交付。"
        "\n\n"
        "你并不亲自完成所有专业工作，而是像真实企业中的项目负责人一样，"
        "根据任务目标协调不同专家 Agent 协作："
        "将文件读取和清洗任务交给文档处理专家，"
        "将风险识别任务交给合同审核专家，"
        "将修改建议和条文改写任务交给合同修改专家，"
        "将最终质量把关和报告归档任务交给终审专家。"
        "\n\n"
        "你的管理原则是："
        "第一，保证任务分工清晰；"
        "第二，保证上下游结果能够衔接；"
        "第三，发现输出不完整时要求补充；"
        "第四，确保最终报告包含关键条款、风险清单、修改建议、前后对比和总体意见；"
        "第五，确保最终文件被正确保存。"
    ),
    verbose=True,
    allow_delegation=True
)


cleaner_agent = Agent(
    role="合同读取与文本清洗专家",
    goal=(
        "准确读取用户指定路径下的合同文件，支持 PDF、TXT、Markdown 等常见格式，"
        "并对合同文本进行专业清洗，输出结构清晰、条款完整、适合后续审查的合同全文。"
    ),
    backstory=(
        "你是一位专业的企业文档处理和合同文本解析专家，长期负责法务系统中的合同文件预处理工作。"
        "你熟悉 PDF 提取文本时可能出现的换行错乱、页眉页脚残留、乱码字符、空格异常、段落丢失等问
题。"
        "\n\n"
        "你的工作不是简单地读取文件，而是要为后续法律审查提供高质量文本基础。"
        "你会尽量保留合同标题、合同主体、条款编号、自然段落、签署信息等重要结构，"
        "避免将整份合同压缩成一整行，因为这会影响后续 Agent 定位具体条款。"
        "\n\n"
        "你需要重点关注："
        "1. 文件是否成功读取；"
        "2. 合同内容是否完整；"
        "3. 条款编号是否保留；"
        "4. 页眉页脚和无意义符号是否被清除；"
        "5. 文本是否适合后续风险识别。"
    ),
    tools=[load_contract_file, clean_contract_text],
    verbose=True,
    allow_delegation=False
)
```

```
reviewer_agent = Agent(
    role="合同法律与商业风险审核专家",
    goal=(
        "基于清洗后的合同文本，系统识别合同中的法律风险、商业风险、合规风险和履约风险，"
        "并按照高、中、低风险等级输出结构化风险点清单。"
    ),
    backstory=(
        "你是一位经验丰富的企业合同审核律师和风险控制专家，熟悉房屋租赁合同、劳务合同、设备租赁合同、"
        "采购合同、服务合同等常见商业合同的审查逻辑。"
        "\n\n"
        "你擅长从企业利益保护角度发现合同中的潜在问题，例如主体信息不完整、付款条件不明确、"
        "验收标准缺失、违约责任不对等、解除条款不公平、保密义务不足、争议解决条款模糊等。"
        "\n\n"
        "你的审查方式应当专业、审慎、结构化。"
        "你不能只笼统地说“存在风险”，而要说明风险出现在哪个条款、原条款怎么写、"
        "为什么有风险、可能造成什么后果、风险等级为什么是高/中/低。"
        "\n\n"
        "你需要重点审查以下维度："
        "1. 合同主体与签约资质；"
        "2. 合同标的、服务范围或租赁物描述；"
        "3. 价款、租金、报酬、付款节点和发票安排；"
        "4. 履行期限、交付时间、验收标准；"
        "5. 双方权利义务是否对等；"
        "6. 违约责任是否明确且可执行；"
        "7. 合同解除、终止和提前退场机制；"
        "8. 保密、知识产权、数据安全等特殊条款；"
        "9. 不可抗力和通知义务；"
        "10. 争议解决方式、管辖法院或仲裁机构。"
    ),
    verbose=True,
    allow_delegation=False
)


modifier_agent = Agent(
    role="合同条款修改与优化专家",
    goal=(
        "针对审核专家识别出的风险点，生成具体、可执行、可直接替换进合同文本的修改建议，"
        "并输出修改前后条文对比表。"
    ),
    backstory=(
        "你是一位资深合同起草与条款优化专家，擅长将风险审核结果转化为可落地的合同修改方案。"
        "你不仅知道某个条款哪里有问题，更擅长写出更严谨、更公平、更保护企业利益的替代条款。"
        "\n\n"
        "你的修改原则包括："
        "1. 保持条款表达清晰、准确、可执行；"
        "2. 平衡法律合规与商业可接受性；"
        "3. 对高风险条款必须给出完整替代文本；"
        "4. 修改后条款应能直接复制进合同；"
        "5. 修改理由要对应具体风险，不写空泛套话。"
        "\n\n"
        "你尤其关注以下优化方向："
```

```
            "付款条件明确化、验收标准具体化、违约责任可量化、解除条件公平化、"
            "保密义务完整化、争议解决条款确定化、通知送达机制清晰化、履约责任边界明确化。"
        ),
        verbose=True,
        allow_delegation=False
)


validator_agent = Agent(
        role="合同审查终审与报告交付专家",
        goal=(
            "对文档清洗、风险审核、修改建议和条文对比结果进行最终复核，"
            "生成完整、专业、结构化的 Markdown 合同审查报告，并保存最终报告和修改后条文文件。"
        ),
        backstory=(
            "你是一位资深企业法务总监，长期负责重大合同的最终审查、风险把关和审查报告签发。"
            "你关注的不只是发现风险，还包括报告是否完整、逻辑是否一致、修改建议是否可落地、"
            "风险等级是否合理、前后条文是否一一对应。"
            "\n\n"
            "你的终审标准非常严格："
            "1. 最终报告必须包含合同基本信息、关键条款提取、风险点清单、修改建议、修改前后对比、总体审查意见；"
            "2. 高风险和中风险问题必须有明确修改建议；"
            "3. 修改前后条文必须能够对应风险编号；"
            "4. 报告语言要专业、克制、适合法务和商务部门阅读；"
            "5. 不得夸大 AI 能力，必须加入 AI 辅助审查声明；"
            "6. 最终必须调用保存工具，生成 contract_review_report.md 和 modified_clauses.md。"
            "\n\n"
            "你是最终交付质量负责人，必须确保输出结果能够作为企业内部合同初审报告使用。"
        ),
        tools=[save_final_report, save_modified_clauses],
        verbose=True,
        allow_delegation=False
)


# ==================== 3. 定义 Task ====================

task1_clean = Task(
        description="""
【任务名称】
合同文件读取与文本清洗

【任务背景】
用户提供了一份企业合同文件，文件路径为：{file_path}。
该合同可能是 PDF、TXT 或 Markdown 格式。后续风险审核、条款修改和终审报告都依赖本阶段输出的文本质量。

【执行要求】
请完成以下工作：
1. 调用 load_contract_file 工具读取合同文件；
2. 检查文件内容是否成功读取；
3. 调用 clean_contract_text 工具清洗合同文本；
4. 输出清洗后的完整合同文本。
```

```
321
322 【清洗标准】
323 - 保留合同标题；
324 - 保留合同主体信息，例如甲方、乙方、出租方、承租方、用工方、劳动者等；
325 - 保留条款编号，例如“第一条”“一、”“1.”“（一）”；
326 - 保留合同主要段落结构；
327 - 删除明显乱码、无意义特殊符号、重复空白；
328 - 删除明显页眉页脚；
329 - 不要将合同压缩成一整行；
330 - 不要擅自改写合同原意；
331 - 不要遗漏签署信息、附件说明、争议解决等后置条款。
332
333 【输出格式】
334 请输出清洗后的合同全文，并在开头简要说明：
335 - 文件是否读取成功；
336 - 原始文本大致长度；
337 - 清洗后文本是否保留条款结构。
338 """,
339     expected_output="""
340 一份清洗后的完整合同文本。
341
342 输出应包含：
343 1. 文件读取状态；
344 2. 清洗说明；
345 3. 清洗后的合同正文。
346
347 合同正文必须保留条款层级和段落结构。
348 """,
349     agent=cleaner_agent
350 )
351
352
353 task2_review = Task(
354     description="""
355 【任务名称】
356 合同法律、商业与合规风险识别
357
358 【任务背景】
359 你将基于上一阶段清洗后的合同文本进行风险审核。
360 本阶段目标不是泛泛总结合同内容，而是识别可操作、可定位、可解释的合同风险点。
361
362 【审查维度】
363 请至少从以下维度进行审查：
364 1. 合同主体是否明确，签约主体是否具备履约能力；
365 2. 合同标的、服务内容、租赁物或工作内容是否具体；
366 3. 合同金额、付款方式、付款节点、发票、押金或保证金是否明确；
367 4. 履行期限、交付时间、服务周期、租赁期限是否清晰；
368 5. 验收标准、交付标准、质量标准是否明确；
369 6. 双方权利义务是否平衡；
370 7. 违约责任是否具体、可执行，是否存在责任缺失；
371 8. 合同解除、终止、提前退租或提前终止机制是否公平；
372 9. 保密、知识产权、数据安全、资料返还等条款是否完整；
373 10. 不可抗力、通知送达、合同变更等一般条款是否完备；
```

```
11. 争议解决方式、管辖法院或仲裁机构是否明确；
12. 是否存在明显不公平、显失公平或表述模糊的条款。

【风险等级定义】
- 高风险：可能导致重大经济损失、重大履约争议、合同核心权利无法保障，或严重不利于企业签约方；
- 中风险：可能造成履约不确定、争议处理成本增加、责任边界不清，但可通过修改条款控制；
- 低风险：主要是表述不够严谨、格式不规范、信息不完整等一般性问题。

【输出要求】
请输出 Markdown 风险点清单。
每个风险点必须包含：
- 风险编号，例如 R-001；
- 风险等级：高 / 中 / 低；
- 条款位置；
- 原条款摘录；
- 风险类型；
- 风险描述；
- 可能后果；
- 初步修改方向。

【注意事项】
- 条款位置应尽量具体，例如“第三条 租金及支付方式”；
- 原条款摘录应尽量来自合同原文；
- 不要编造合同中不存在的条款；
- 如果合同缺少某类重要条款，也可以作为风险点，但要说明“缺失风险”。
""",
    expected_output="""
结构化风险点清单，必须使用 Markdown 表格。

表格字段如下：
| 风险编号 | 风险等级 | 条款位置 | 原条款摘录 | 风险类型 | 风险描述 | 可能后果 | 初步修改方向 |
|---|---|---|---|---|---|---|---|

风险点应覆盖法律风险、商业风险和合规风险。
""",
    agent=reviewer_agent,
    context=[task1_clean]
)


task3_modify = Task(
    description="""
【任务名称】
合同修改建议与条文优化

【任务背景】
你将基于风险审核专家输出的风险点清单，逐项提出修改建议。
本阶段的核心目标是将“风险识别”转化为“可直接执行的合同修改方案”。

【执行要求】
请完成以下工作：
1. 阅读风险点清单，识别每个风险编号对应的问题；
2. 为每个高风险和中风险问题提供明确修改建议；
```

```
3. 对低风险问题给出简洁优化建议；
4. 针对需要修改的条款，提供“原条款”和“修改后条款”的对比；
5. 修改后条款必须完整、清晰、可直接替换进合同。

【修改原则】
- 不得只写“建议完善”“建议明确”等空泛内容；
- 修改后条款应尽量使用正式合同语言；
- 修改理由必须对应具体风险；
- 对付款、违约、解除、验收、争议解决等核心条款，要体现可执行性；
- 对缺失条款，可以提供“建议新增条款”；
- 条款修改应兼顾企业利益保护和商业合理性。

【输出结构】
请输出两个部分：

第一部分：修改建议列表

字段包括：
| 建议编号 | 对应风险 | 修改方向 | 具体建议 | 修改理由 |
|---|---|---|---|---|

第二部分：修改前后的条文对比

字段包括：
| 风险编号 | 条款位置 | 原条款 | 修改后条款 | 修改理由 |
|---|---|---|---|---|

【特别要求】
- 对高风险条款必须提供完整修改后条款；
- 对合同缺失的关键条款，应在“原条款”中写明“合同未约定”，并在“修改后条款”中提供建议新增条款；
- 修改后条款不要过短，要具备实际合同可用性。
""",
    expected_output="""
修改建议列表 + 修改前后的条文对比表。

必须包含：

一、修改建议列表
| 建议编号 | 对应风险 | 修改方向 | 具体建议 | 修改理由 |

二、修改前后的条文对比
| 风险编号 | 条款位置 | 原条款 | 修改后条款 | 修改理由 |

所有高风险和中风险问题必须有对应修改方案。
""",
    agent=modifier_agent,
    context=[task2_review]
)


task4_validate = Task(
    description="""
【任务名称】
```

```
终审复核与合同审查报告生成

【任务背景】
你将基于以下三个阶段的结果进行最终复核：
1. 合同读取与清洗结果；
2. 合同风险识别结果；
3. 合同修改建议与条文对比结果。

你的任务不是简单拼接内容，而是作为最终质量负责人，对报告进行专业化整合、检查和交付。

【终审检查要求】
请重点检查：
1. 是否提取了合同基本信息；
2. 是否识别了关键条款；
3. 风险点是否有条款位置、风险等级和风险描述；
4. 高风险和中风险是否都有修改建议；
5. 修改前后条文是否一一对应；
6. 修改建议是否可执行；
7. 报告结构是否完整；
8. 语言是否专业、清晰、适合法务和商务阅读；
9. 是否加入 AI 辅助审查声明；
10. 是否调用保存工具保存最终文件。

【最终报告格式】
请严格按照以下 Markdown 结构生成报告：

# 企业合同智能审查报告

## 一、合同基本信息
- 合同名称：
- 合同类型：
- 合同主体：
- 审查日期：
- 审查方式：CrewAI Hierarchical 多 Agent 智能审查

## 二、关键条款提取表
| 条款类别 | 条款位置 | 主要内容 | 审查关注点 |
|---|---|---|---|

至少提取以下类别：
- 合同主体；
- 合同标的；
- 合同金额或费用；
- 履行期限；
- 付款或结算方式；
- 违约责任；
- 解除或终止；
- 争议解决。

## 三、风险点清单
| 风险编号 | 风险等级 | 条款位置 | 风险类型 | 风险描述 | 可能后果 |
|---|---|---|---|---|---|

```

```
## 四、修改建议列表
| 建议编号 | 对应风险 | 修改方向 | 具体建议 | 修改理由 |
|---|---|---|---|---|

## 五、修改前后的条文对比
| 风险编号 | 条款位置 | 原条款 | 修改后条款 | 修改理由 |
|---|---|---|---|---|

这是本项目的重点部分，必须完整、清晰、可读。

## 六、总体审查意见
### 1. 总体风险等级
请根据风险点数量、风险严重程度和合同核心条款完整性，判断总体风险等级：高 / 中 / 低。

### 2. 主要问题概述
请概括本合同最主要的 3 到 5 类问题。

### 3. 签署前建议
请给出企业签署前应采取的操作建议。

## 七、修改后条文汇总
请单独整理所有“修改后条款”或“建议新增条款”。

## 八、AI 辅助审查声明
本报告由 AI 辅助生成，仅供企业内部初步审查参考，不构成正式法律意见。正式签署前，建议由专业律师或企业法务人员进行最终确认。

【文件保存要求】
生成最终报告后，必须完成：
1. 调用 save_final_report 工具，将完整报告保存为 contract_review_report.md；
2. 调用 save_modified_clauses 工具，将“修改后条文汇总”保存为 modified_clauses.md。

【注意事项】
- 不要输出空表；
- 不要遗漏“修改前后的条文对比”；
- 如果某些信息合同中未明确写出，应标注“合同未明确约定”，不要编造；
- 报告语言要专业、克制、结构化。
""",
    expected_output="""
完整的企业合同智能审查报告，Markdown 格式。

必须确认以下内容：
1. 已生成关键条款提取表；
2. 已生成风险点清单；
3. 已生成修改建议列表；
4. 已生成修改前后的条文对比；
5. 已生成总体审查意见；
6. 已保存 outputs/contract_review_report.md；
7. 已保存 outputs/modified_clauses.md。
""",
    agent=validator_agent,
    context=[task1_clean, task2_review, task3_modify]
)
```

```python


# ==================== 4. 组建 Crew（Hierarchical 模式） ====================

crew = Crew(
    agents=[
        cleaner_agent,
        reviewer_agent,
        modifier_agent,
        validator_agent
    ],
    tasks=[
        task1_clean,
        task2_review,
        task3_modify,
        task4_validate
    ],
    process=Process.hierarchical,
    manager_agent=manager_agent,
    verbose=True
)


# ==================== 5. 执行项目 ====================

if __name__ == "__main__":
    print("企业合同智能审查多 Agent 系统启动：Hierarchical 模式\n")

    # 示例文件路径，可根据实际附件修改：
    # contract_path = "contracts/房屋租赁合同.txt"
    # contract_path = "contracts/劳务合同.md"
    # contract_path = "contracts/设备租赁合同.pdf"
    contract_path = "contract_sample.pdf"

    result = crew.kickoff(inputs={"file_path": contract_path})

    print("\n" + "=" * 80)
    print("最终审查报告")
    print("=" * 80)
    print(result)

    print("\n✅ 运行结束，请查看 outputs 目录：")
    print("- outputs/contract_review_report.md")
    print("- outputs/modified_clauses.md")
```

附件：合同包.zip

包括：

房屋租赁合同.txt

劳务合同.md

设备租赁合同.pdf

3. 项目开发补救与效率提升：如何用 AI 辅助完成复杂项目

在学习 Multi-Agent 项目时，真正重要的不只是“代码能不能一次跑通”，而是能否理解项目结构、定位问题来源、快速修复 bug，并逐步提升项目质量。尤其是 CrewAI、LangChain、大模型 API 这类技术更新很快，学生在自己完成项目时遇到报错、版本不兼容、Agent 输出不稳定，都是非常正常的现象。

本节重点不是讲“如何避免所有 bug”，而是讲**遇到问题后如何高效处理。**做 AI 项目，要学会把 AI 当成开发助手，用它帮助我们分析错误、优化代码、完善提示词、生成测试用例和补充项目文档。这是现代 AI 开发者必须具备的能力。

3.1 正确认识复杂 AI 项目中的 Bug

复杂 AI 项目和普通 Python 小练习不同，它涉及模型调用、工具调用、Agent 协作、上下文传递、文件解析、依赖版本等多个环节。因此，项目运行过程中出现问题是正常现象。

3.1.1 为什么 AI Agent 项目更容易出现 Bug

1. 技术栈多，依赖复杂

一个 Multi-Agent 项目通常不只依赖 Python 本身，还可能涉及：

- `crewai`
- `crewai-tools`
- `langchain`
- `langchain-community`
- `openai`
- `pymupdf`
- `python-dotenv`
- 本地文件系统
- 大模型 API 服务

这些库之间存在版本依赖关系，只要其中某个库更新，就可能导致原来的代码出现参数不兼容、方法变更或导入失败等问题。

例如：

```
verbose=2
```

在某些版本中可能可以使用，但在新版本中可能要求写成：

```
verbose=True
```

因此，报错不一定代表项目思路错误，很多时候只是**版本适配问题。**

2. Agent 框架版本更新很快

CrewAI、LangChain 等 Agent 框架仍在快速迭代中，很多 API 写法会随着版本变化而调整。

常见变化包括：

- Agent 参数变化；
- Task 参数变化；
- Tool 定义方式变化；
- Manager Agent 配置方式变化；
- Hierarchical 流程要求变化；
- LLM 配置方式变化；
- verbose 参数类型变化。

所以学生在复现项目时，可能会遇到：

```
这个代码老师能跑，我这里为什么报错？
```

这很常见。原因可能是：

- 老师之前使用的是旧版本；
- 学生安装的是新版本；
- 不同环境中的依赖版本不一致；
- 某些参数在新版本中已经调整。

3. 大模型输出具有不确定性

传统程序通常是确定性的：

```
输入 A → 输出 B
```

但 Agent 项目依赖大模型，大模型具有一定不确定性：

```
同样输入 → 可能生成不同表达、不同格式、不同执行路径
```

例如：

- 有时 Agent 会严格输出 Markdown 表格；
- 有时可能漏掉某个字段；
- 有时没有主动调用工具；
- 有时风险分析过于简略；
- 有时修改建议不够具体。

这类问题不是单纯代码 bug，而是 **LLM 输出控制问题**，需要通过更明确的 Task 描述、更严格的输出模板、更好的 Validator Agent 来改善。

4. Hierarchical 模式比 Sequential 模式更复杂

Sequential 模式是顺序执行：

```
Task 1 → Task 2 → Task 3 → Task 4
```

Hierarchical 模式中增加了 Manager Agent：

```
Manager Agent → 分配任务 → 专业 Agent 执行
```

因此它多了一个调度层，可能出现：

- Manager 没有正确分配任务；
- Manager 没有设置 `allow_delegation=True`；
- Manager 被错误放入普通 agents 列表；
- 当前版本要求设置 `manager_llm`；
- Manager 与普通 Agent 职责冲突；
- Manager 试图自己完成所有任务。

所以 Hierarchical 模式更适合作为进阶项目，也更需要学生具备调试意识。

3.2 遇到 Bug 时的正确心态

做项目时，报错不是失败，而是工程能力提升的入口。真正重要的是：看到报错以后，能不能冷静判断问题属于哪一层。

3.2.1 不要把报错理解成“我不会”

1. 报错是项目开发的正常环节

在做项目时，最容易出现的心理状态是：

```
一报错就觉得自己不适合写代码。
```

这是不对的。真实开发中，工程师每天都在处理各种 bug。尤其是 AI 项目，外部依赖多、框架变化快、模型输出不稳定，报错更加常见。更合理的心态是：

```
报错不是说明我不会，而是说明我进入了真实开发阶段。
```

2. 项目能力体现在调试过程中

会复制一份代码并不代表真正掌握项目。真正的项目能力体现在：

- 能读懂报错；
- 能判断错误来源；
- 能定位是哪一段代码的问题；

- 能查文档；
- 能问 AI；
- 能根据建议修改代码；
- 能验证修改是否有效；
- 能总结成自己的经验。

所以不要追求“永远不报错”，而要追求：

```
报错以后，我知道怎么一步步排查。
```

3. 不要害怕问 AI

现在学习 AI 项目，就应该学会使用 AI 工具提高开发效率。

遇到问题时，可以把：

- 报错信息；
- 相关代码；
- 当前环境；
- 使用的库版本；
- 你想实现的目标；

一起发给 AI，让 AI 帮你分析。

这不是偷懒，而是现代开发方式。

关键是：

```
不是让 AI 替你学习，而是让 AI 帮你提高定位问题的效率。
```

3.3 Bug 排查的基本流程

当项目运行失败时，不要随机改代码。建议按照固定流程排查。

3.3.1 第一步：先看错误类型

1. 判断错误属于哪一层

常见错误可以分为六类：

错误类型	常见表现	排查方向
环境配置问题	API Key 缺失、模型不可用	检查 `.env`、模型配置、网络
依赖版本问题	参数不兼容、模块导入失败	检查库版本，升级或降级依赖
文件路径问题	找不到合同文件	检查文件是否存在、路径是否正确

错误类型	常见表现	排查方向
工具调用问题	Tool 没被调用、读取失败	检查工具是否绑定给对应 Agent
Manager 配置问题	hierarchical 报错、Manager 不工作	检查 `allow_delegation`、`manager_agent`
输出格式问题	表格缺失、报告不完整	优化 Prompt 和 expected_output

不要看到报错就直接修改业务代码，要先判断它属于哪一层。

3.3.2 第二步：检查运行环境

1. 检查 Python 版本

建议使用相对稳定的 Python 版本，例如：

```
Python 3.10 或 Python 3.11
```

可以使用命令查看：

```
python --version
```

2. 检查依赖是否安装

常见依赖包括：

```
pip install crewai crewai-tools langchain-community pymupdf python-dotenv
```

如果使用 OpenAI 相关模型，还可能需要：

```
pip install openai langchain-openai
```

3. 检查依赖版本

查看某个库版本：

```
pip show crewai
```

或：

```
pip freeze
```

如果发现老师代码和自己环境不一致，可以把版本信息发给 AI，让 AI 帮你判断是否存在兼容问题。

3.3.3 第三步：检查文件路径

1. 文件路径错误是最常见问题之一

例如代码中写的是：

```
contract_path = "contract_sample.pdf"
```

但实际文件在：

```
contracts/设备租赁合同.pdf
```

就会出现文件读取失败。

2. 推荐做法

把文件统一放到 `contracts` 文件夹下：

```
contracts/
├── 房屋租赁合同.txt
├── 劳务合同.md
└── 设备租赁合同.pdf
```

然后在代码中写：

```
contract_path = "contracts/设备租赁合同.pdf"
```

3. 检查文件是否存在

可以临时加入：

```
import os

print(os.path.exists(contract_path))
print(os.getcwd())
```

如果输出：

```
False
```

说明当前运行目录和文件路径没有对上。

3.3.4 第四步：检查 Manager 配置

1. Hierarchical 模式必须正确配置 Manager

本项目使用：

```
process=Process.hierarchical
```

因此 Manager Agent 是核心角色。

正确配置应包括：

```
allow_delegation=True
```

例如：

```
manager_agent = Agent(
    role="企业合同智能审查项目经理",
    goal="统筹合同审查流程，协调各专业 Agent 完成任务。",
    backstory="你是一位经验丰富的合同审查项目经理。",
    verbose=True,
    allow_delegation=True
)
```

2. Manager 应通过 `manager_agent` 单独传入

推荐写法：

```
crew = Crew(
    agents=[
        cleaner_agent,
        reviewer_agent,
        modifier_agent,
        validator_agent
    ],
    tasks=[
        task1_clean,
        task2_review,
        task3_modify,
        task4_validate
    ],
    process=Process.hierarchical,
    manager_agent=manager_agent,
    verbose=True
)
```

注意：不要把 Manager 重复放入普通 Agent 列表：

```
agents=[
    manager_agent,
    cleaner_agent,
    reviewer_agent,
    modifier_agent,
    validator_agent
]
```

这样可能导致 Manager 既是管理者，又被当成普通执行者。

3. 某些版本可能要求配置 `manager_llm`

如果报错类似：

```
Manager agent should have llm
```

可以尝试显式配置 LLM。

示例：

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.2
)
```

然后在 Manager Agent 中加入：

```
llm=llm
```

3.3.5 第五步：检查 Tool 是否绑定正确

1. 工具必须绑定给需要使用它的 Agent

例如，读取文件和清洗文本的工具应该绑定给 Cleaner Agent：

```
cleaner_agent = Agent(
    role="合同读取与文本清洗专家",
    tools=[load_contract_file, clean_contract_text],
    ...
)
```

保存报告的工具应该绑定给 Validator Agent：

```
validator_agent = Agent(
    role="合同审查终审与报告交付专家",
    tools=[save_final_report, save_modified_clauses],
    ...
)
```

如果工具没有绑定给对应 Agent，Agent 就无法调用该工具。

2. Task 中要明确要求调用工具

例如：

```
请调用 load_contract_file 工具读取合同文件；
请调用 clean_contract_text 工具清洗合同文本；
```

这样可以提高 Agent 正确调用工具的概率。

3.3.6 第六步：检查 Context 是否正确传递

1. 后续任务依赖前序任务结果

例如：

```
task2_review = Task(
    ...,
    context=[task1_clean]
)
```

表示风险审核任务依赖清洗后的合同文本。

```
task3_modify = Task(
    ...,
    context=[task2_review]
)
```

表示修改建议任务依赖风险点清单。

```
task4_validate = Task(
    ...,
    context=[task1_clean, task2_review, task3_modify]
)
```

表示终审任务需要综合前面所有结果。

2. 如果 Context 缺失，Agent 可能无法完成任务

例如 Modifier Agent 如果没有拿到 Reviewer Agent 的风险清单，就可能只能泛泛生成修改建议。

因此要检查：

- `context` 是否写了；

- 上一个 Task 是否有明确输出；
- 下一个 Task 是否能理解上一个 Task 的输出结构。

3.4 如何使用 AI 辅助修复 Bug

学习 AI 项目时，要学会把 AI 当作开发助手。遇到 bug 时，可以让 AI 帮你分析错误、解释代码、生成修复方案。

3.4.1 向 AI 提问的正确方式

1. 不推荐的提问方式

不要只问：

```
代码报错了，怎么办？
```

这种问题信息太少，AI 很难准确判断。

2. 推荐的提问方式

应该提供完整上下文：

```
我正在做一个 CrewAI 多 Agent 项目，使用 Process.hierarchical 模式。
项目中有 Manager Agent、Cleaner Agent、Reviewer Agent、Modifier Agent 和 Validator Agent。
Manager Agent 设置了 allow_delegation=True，并通过 manager_agent 参数传入 Crew。

现在运行时报错如下：
【粘贴完整报错】

相关代码如下：
【粘贴 manager_agent 和 crew 配置代码】

我的 crewai 版本是：
【粘贴 pip show crewai 的结果】

请帮我分析：
1. 这个错误最可能是什么原因？
2. 是否和 CrewAI 版本有关？
3. 如果是 Manager 配置问题，应该如何修改？
4. 请给出修改后的代码片段。
```

这样 AI 才能根据完整信息给出有效建议。

3.4.2 通用 AI 修 Bug 提示词模板

1. 版本兼容问题模板

```
我正在运行一个 CrewAI 项目，代码可能是基于旧版本写的。
当前安装的新版本运行时报错。

我的目标：
使用 Process.hierarchical 构建多 Agent 系统，并通过 manager_agent 配置 Manager Agent。

当前报错：
【粘贴完整报错】

相关代码：
【粘贴 Crew、Agent、Task 相关代码】

当前版本：
【粘贴 pip show crewai 或 pip freeze 中相关版本】

请你帮我：
1. 判断是否是 CrewAI 版本兼容问题；
2. 找出可能不兼容的参数或写法；
3. 给出适配当前版本的修改代码；
4. 解释每一处修改原因。
```

2. Manager 配置问题模板

```
我正在使用 CrewAI 的 Process.hierarchical 模式。
项目中配置了一个 Manager Agent，用于分配任务，但运行时 Manager 没有正常工作或报错。

Manager Agent 代码：
【粘贴 manager_agent 代码】

Crew 配置代码：
【粘贴 crew = Crew(...) 代码】

报错信息或异常表现：
【粘贴报错，或描述 Manager 没有分配任务】

请帮我检查：
1. Manager Agent 是否应该设置 allow_delegation=True；
2. Manager 是否应该单独通过 manager_agent 参数传入；
3. Manager 是否不应该放入普通 agents 列表；
4. 当前版本是否需要 manager_llm；
5. 请给出推荐修复后的代码。
```

3. Tool 调用问题模板

```
我在 CrewAI 项目中定义了工具，但 Agent 没有正确调用工具。

工具代码：
【粘贴 @tool 定义】
```

```
Agent 代码：
【粘贴绑定 tools 的 Agent】

Task 描述：
【粘贴相关 Task】

运行表现：
【例如：没有读取文件 / 没有保存报告 / 工具没有被调用】

请帮我分析：
1. 工具定义是否正确；
2. 工具是否绑定到了正确 Agent；
3. Task 描述是否需要更明确要求调用工具；
4. 是否需要修改工具参数或返回值；
5. 请给出修改后的代码。
```

4. 文件路径问题模板

```
我的 CrewAI 项目需要读取本地合同文件，但运行时提示文件不存在或读取失败。

当前代码：
contract_path = "【你的路径】"

项目目录结构：
【粘贴你的目录结构】

当前运行目录：
【粘贴 os.getcwd() 的输出】

报错信息：
【粘贴完整报错】

请帮我判断：
1. 文件路径是否写错；
2. 是否应该使用相对路径或绝对路径；
3. 如何检查当前工作目录；
4. 请给出推荐修改方式。
```

5. 输出格式不稳定模板

```
我正在使用 CrewAI 多 Agent 生成合同审查报告。
问题是模型输出格式不稳定，有时没有 Markdown 表格，有时缺少“修改前后的条文对比”。

当前 Task 描述：
【粘贴 task description】

当前 expected_output：
【粘贴 expected_output】

我希望最终输出包含：
1. 关键条款提取表；
```

```
2. 风险点清单；
3. 修改建议列表；
4. 修改前后的条文对比；
5. 总体审查意见。

请帮我优化 Task Prompt：
1. 让输出结构更稳定；
2. 强制使用 Markdown 表格；
3. 明确每个表格字段；
4. 增加质量检查要求；
5. 给出优化后的 Task description 和 expected_output。
```

3.5 如何提高项目开发效率

做复杂项目时，不能只靠一次性写完整代码。更高效的方法是分阶段开发、逐步验证。

3.5.1 先跑通最小版本

1. 不要一开始就追求完整系统

建议先跑通最小链路：

```
读取文件 → 清洗文本 → 打印结果
```

确认文件读取没有问题后，再加入风险审核 Agent。

然后逐步扩展：

```
读取清洗 → 风险审核 → 修改建议 → 终审报告 → 文件保存
```

2. 分阶段验证每个 Agent

每增加一个 Agent，就单独检查它的输入输出：

- Cleaner 是否读到了合同；
- Reviewer 是否识别出了风险；
- Modifier 是否生成了修改建议；
- Validator 是否保存了报告；
- Manager 是否正常协调任务。

这样比一次性运行完整项目更容易定位问题。

3.5.2 保留可运行版本

1. 每次大改前先备份

例如：

```
main_v1_sequential.py
main_v2_hierarchical.py
main_v3_with_manager.py
main_v4_final.py
```

这样如果新版本报错，可以随时退回上一版。

2. 不要在唯一文件上反复乱改

很多学生项目最后跑不起来，是因为：

- 改了太多地方；
- 不知道是哪一步改坏的；
- 没有保留之前能运行的版本。

建议养成习惯：

```
每跑通一个阶段，就保存一个版本。
```

3.5.3 使用日志和中间输出

1. 打印关键变量

例如：

```
print("当前合同路径：", contract_path)
print("文件是否存在：", os.path.exists(contract_path))
```

2. 保存中间结果

可以把清洗后的合同文本先保存出来：

```
with open("outputs/cleaned_contract.txt", "w", encoding="utf-8") as f:
    f.write(cleaned_text)
```

这样可以判断问题是在：

- 文件读取阶段；
- 文本清洗阶段；
- 风险审核阶段；
- 报告生成阶段。

3. 开启 verbose

Agent 执行时建议开启：

```
verbose=True
```

这样可以看到 Agent 的执行过程，有助于调试。

3.6 如何优化提示词，提高输出质量

Agent 项目的输出质量很大程度上取决于 Prompt 和 Task 设计。

3.6.1 提示词不要太短

1. 简单提示词容易导致输出泛泛

例如：

```
请审查合同风险。
```

这个提示太短，模型可能只输出几条笼统建议。

更好的写法是：

```
请从合同主体、付款方式、履行期限、验收标准、违约责任、解除条款、保密条款、争议解决等维度审查合同风险。每个风险点必须包含风险编号、风险等级、条款位置、原条款摘录、风险描述、可能后果和修改方向。
```

这样输出会明显更稳定。

2. Task 中要明确输出格式

推荐使用表格模板：

```
| 风险编号 | 风险等级 | 条款位置 | 原条款摘录 | 风险类型 | 风险描述 | 可能后果 |
|---|---|---|---|---|---|---|
```

模型看到明确表格结构，更容易按格式输出。

3.6.2 使用“角色 + 目标 + 标准 + 格式”

1. 一个好的 Agent Task 应包含四部分

组成部分	作用
角色	告诉 Agent 它是谁
目标	告诉 Agent 要完成什么
标准	告诉 Agent 什么叫完成得好

组成部分	作用
格式	告诉 Agent 怎么输出

例如：

```
你是一位企业合同审核律师。
你的目标是识别合同中的法律、商业和合规风险。
请按照合同主体、付款、履行、违约、解除、争议解决等维度审查。
输出必须使用 Markdown 表格，字段包括风险编号、风险等级、条款位置、风险描述和可能后果。
```

2. 对重点输出要反复强调

本项目重点是：

```
修改前后的条文对比
```

因此可以在 Task 中多次强调：

- 必须输出修改前后对比；
- 必须使用 Markdown 表格；
- 修改后条款必须可直接替换；
- 高风险条款必须有修改后条款。

3.7 推荐使用的 AI 协作开发流程

3.7.1 第一步：让 AI 帮你解释代码

```
请帮我逐段解释下面这个 CrewAI 多 Agent 项目代码。
重点解释：
1. 每个 Agent 的作用；
2. 每个 Task 的作用；
3. Tools 是如何被调用的；
4. context 是如何传递的；
5. hierarchical 模式下 Manager Agent 的作用。

代码如下：
【粘贴代码】
```

3.7.2 第二步：让 AI 帮你找潜在问题

```
请帮我检查下面这个 CrewAI 项目代码是否存在潜在问题。
重点检查：
1. Manager Agent 是否配置正确；
2. Tools 是否绑定到正确 Agent；
3. Task 是否正确使用 context；
4. file_path 是否能正确传入；
5. 是否存在版本兼容风险；
6. 输出文件是否能正常保存。

代码如下：
【粘贴代码】
```

3.7.3 第三步：让 AI 帮你优化 Prompt

```
我正在做一个企业合同智能审查多 Agent 项目。
请帮我优化以下 Agent 和 Task 的提示词，使其更专业、更适合写入简历，并提高输出稳定性。

要求：
1. 每个 Agent 的 role、goal、backstory 更详细；
2. 每个 Task 的 description 包含任务背景、执行步骤、输出格式和质量标准；
3. 输出必须包含 Markdown 表格；
4. 重点强化“修改前后的条文对比”。

当前代码：
【粘贴 Agent 和 Task 代码】
```

3.7.4 第四步：让 AI 帮你生成 README

```
请根据下面这个 CrewAI 合同智能审查多 Agent 项目，帮我生成一份 README.md。

README 需要包含：
1. 项目背景；
2. 项目功能；
3. 技术栈；
4. Agent 分工；
5. 系统流程图；
6. 安装依赖；
7. 运行方式；
8. 输入输出说明；
9. 示例输出；
10. 项目亮点；
11. 可优化方向。

项目代码如下：
【粘贴代码】
```

3.7.5 第五步：让 AI 帮你写简历描述

```
请帮我把下面这个 CrewAI 多 Agent 项目整理成适合写入简历的项目经历。

要求包含：
1. 项目名称；
2. 项目背景；
3. 技术栈；
4. 我的工作内容；
5. 项目亮点；
6. 技术难点；
7. 项目成果；
8. 适合面试表达的话术。

项目内容：
【粘贴项目介绍或代码】
```

3.8 项目优化方向

当基础项目完成后，可以继续优化，让项目更有深度。

3.8.1 工程稳定性优化

1. 固定依赖版本

创建 `requirements.txt`：

```
crewai
crewai-tools
langchain-community
pymupdf
python-dotenv
openai
langchain-openai
```

更严谨的方式是固定版本号：

```
crewai==指定版本
langchain-community==指定版本
pymupdf==指定版本
```

这样可以减少环境差异导致的问题。

2. 增加异常处理

例如：

- 文件不存在时给出明确提示；
- PDF 解析失败时建议切换 TXT；
- 保存失败时检查目录权限；
- API 调用失败时提示检查 Key 和网络。

3. 增加兜底保存逻辑

即使 Validator Agent 没有调用保存工具，主程序也可以保存结果：

```
os.makedirs("outputs", exist_ok=True)

with open("outputs/contract_review_report.md", "w", encoding="utf-8") as f:
    f.write(str(result))
```

3.8.2 输出质量优化

1. 增加报告检查 Agent

可以新增一个 Report Checker Agent，专门检查：

- 是否有关键条款提取表；
- 是否有风险点清单；
- 是否有修改建议；
- 是否有修改前后对比；
- 是否有总体审查意见；
- 是否有 AI 辅助声明。

2. 使用结构化输出

后续可以结合 Pydantic 或 JSON Schema，让风险点和修改建议更稳定。

3. 增加合同类型识别 Agent

在 Cleaner 之后新增一个 Agent：

```
Contract Classifier Agent
```

用于判断合同类型：

- 房屋租赁合同；
- 劳务合同；
- 设备租赁合同；
- 服务合同；
- 采购合同。

然后根据合同类型使用不同审查维度。

3.9 做项目时应掌握的核心能力

3.9.1 看懂架构

要能说明：

- 为什么使用 Multi-Agent；
- 为什么使用 Manager；
- 每个 Agent 负责什么；
- Tools 如何扩展能力；
- Context 如何传递结果。

3.9.2 会排查 Bug

要能判断问题属于：

- 环境问题；
- 依赖问题；
- 文件路径问题；
- Manager 配置问题；
- Tool 调用问题；
- 输出格式问题。

3.9.3 会使用 AI 提升效率

要善于让 AI 帮助：

- 解释代码；
- 分析报错；
- 修改 bug；
- 优化 Prompt；
- 生成 README；
- 总结简历项目；
- 准备答辩话术。

3.9.4 保持正确心态

复杂项目不可能永远一次跑通。

真正的成长来自：

```
遇到问题 → 定位问题 → 使用工具 → 修改验证 → 总结经验
```

这才是从“会运行代码”到“会做项目”的关键转变。